

实验报告

练习1: 加载应用程序并执行

设计实现过程

在 `load_icode` 函数中，我们需要设置新进程的 `trapframe`，以便在内核返回用户态时，处理器能够正确地跳转到应用程序的入口点，并设置正确的栈指针和特权级。

具体实现如下（位于 `kern/process/proc.c` 的 `load_icode` 函数中）：

```
tf->gpr.sp = USTACKTOP;
tf->epc = elf->e_entry;
tf->status = (read_csr(sstatus) & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

1. 设置栈指针 (`tf->gpr.sp`): 将用户栈指针设置为 `USTACKTOP`。这是用户地址空间中栈的顶部。
2. 设置程序计数器 (`tf->epc`): 将异常程序计数器设置为 ELF 可执行文件的入口地址 (`elf->e_entry`)。当执行 `sret` 指令从内核态返回时，PC 将跳转到这个地址。
3. 设置状态寄存器 (`tf->status`):
 - `read_csr(sstatus) & ~SSTATUS_SPP`: 清除 `SPP` (Supervisor Previous Privilege) 位。`SPP` 位决定了从 S 模式返回时进入的特权级。将其设置为 0 表示返回到 User 模式。
 - `| SSTATUS_SPIE`: 设置 `SPIE` (Supervisor Previous Interrupt Enable) 位。这确保了在返回用户态后，中断是被允许的。

用户态进程执行流程描述

当这个用户态进程被 uCore 选择占用 CPU 执行（RUNNING 态）到具体执行应用程序第一条指令的整个经过如下：

1. 调度器选择: `schedule` 函数从运行队列中选择该进程，并调用 `proc_run`。
2. 上下文切换: `proc_run` 调用 `switch_to` 函数，保存当前进程的上下文（寄存器状态），并恢复新进程的上下文。

3. 内核入口: 新进程的上下文设置其返回地址 (ra) 为 `forkret`。因此, `switch_to` 返回后, 执行流跳转到 `forkret`。
4. 中断返回准备: `forkret` 调用 `forkrets`, 并将当前进程的 `trapframe` 指针作为参数传递。
5. 恢复寄存器: `forkrets` (通常在 `trapentry.S` 中实现) 将 `trapframe` 中的内容恢复到 CPU 寄存器中。这包括通用寄存器、`sepc` (设置为应用程序入口)、`sstatus` (设置为用户模式且开启中断)。
6. 执行 `sret`: 执行 `sret` 指令。CPU 根据 `sstatus.SPP` 切换到用户模式 (U-mode), 根据 `sstatus.SPIE` 恢复中断使能状态, 并将 PC 跳转到 `sepc` 指向的地址 (即应用程序的第一条指令)。
7. 执行应用: 应用程序开始执行。

练习2: 父进程复制自己的内存空间给子进程

设计实现过程

在 `copy_range` 函数中, 我们实现了内存的复制。为了支持 Copy-on-Write (COW) 机制, 我们需要根据 `share` 参数来决定是进行物理页的复制还是共享。

具体实现如下 (位于 `kern/mm/pmm.c` 的 `copy_range` 函数中):

```
if (share)
{
    // COW implementation
    if (perm & PTE_W)
    {
        // 如果页面是可写的, 清除写权限并设置 COW 标志
        uint32_t cow_perm = (perm & ~PTE_W) | PTE_COW;

        // 更新父进程的 PTE
        *ptep = pte_create(page2ppn(page), cow_perm);

        // 为子进程创建 PTE, 共享同一个物理页
        *nptep = pte_create(page2ppn(page), cow_perm);

        // 增加物理页的引用计数
        page_ref_inc(page);
    }
    else
    {
        // 如果页面本身只读, 直接共享, 不需要设置 COW
```

```

        *nptep = pte_create(page2ppn(page), perm);
        page_ref_inc(page);
    }
}
else
{
    // 非共享模式（原有实现）：分配新页并复制内容
    struct Page *npage = alloc_page();
    assert(npage != NULL);
    int ret = 0;
    void *src_kvaddr = page2kva(page);
    void *dst_kvaddr = page2kva(npage);
    memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
    ret = page_insert(to, npage, start, perm);
    assert(ret == 0);
}

```

Copy on Write (COW) 机制设计

概要设计：

Copy-on-Write 的核心思想是推迟物理内存的复制操作，直到真正需要写入时才进行。

1. **Fork 阶段**: 当父进程创建子进程时，不立即复制物理内存。而是将父子进程的虚拟页都映射到同一个物理页上。
2. **权限设置**: 将这些共享页面的页表项 (PTE) 设置为只读，并标记为 **COW** (Copy-on-Write)。
3. **写操作触发**: 当任一进程尝试写入这些只读页面时，CPU 会触发页访问异常 (Page Fault)。
4. **异常处理**:
 - 内核的页错误处理程序 (`do_pgfault`) 捕获异常。
 - 检查异常原因是否为写操作，且该页面是否标记为 COW。
 - 如果是，内核分配一个新的物理页。
 - 将原物理页的内容复制到新物理页。
 - 更新当前进程的页表，将虚拟页映射到这个新物理页，并将权限修改为可写，清除 COW 标志。
 - 减少原物理页的引用计数。
5. **恢复执行**: 异常处理返回，重新执行刚才触发异常的写指令，此时写入将成功。

详细设计：

- **PTE 标志位**: 在 `kern/mm/mmu.h` 中定义 `PTE_COW` (例如使用 RSW 位)。

- 引用计数: 利用 `struct Page` 中的 `ref` 成员来跟踪物理页被多少个进程共享。
- `do_fork`: 调用 `copy_mm -> dup_mmap -> copy_range`。在 `copy_range` 中, 如果 `share=1`, 则执行上述的共享逻辑 (设置只读、设置 COW、增加引用计数)。注意需要刷新 TLB。
- `do_pgfault`: 在 `kern/mm/vmm.c` 中。
 - 当发生写异常 (`error_code & 1`) 时, 检查 PTE。
 - 如果 `*ptep & PTE_COW` 为真:
 - 如果 `page_ref(page) > 1`: 说明还有其他进程共享此页。分配新页, 拷贝内容, 建立新映射 (可写), 原页引用计数减 1。
 - 如果 `page_ref(page) == 1`: 说明只有当前进程引用此页 (其他进程可能已经 COW 分离了, 或者已经退出了)。直接将当前 PTE 修改为可写, 清除 COW 标志, 不需要分配新页。

练习3: 阅读分析源代码

fork/exec/wait/exit 执行流程分析

1. fork:

- 用户态: 调用 `fork()` 库函数, 最终执行 `ecall` 指令陷入内核。
- 内核态: `sys_fork -> do_fork`。
 - 分配 `proc_struct`。
 - 分配内核栈。
 - `copy_mm`: 复制或共享内存空间 (COW 在此发生)。
 - `copy_thread`: 设置子进程的 `trapframe` 和上下文。子进程的 `tf->gpr.a0` 被置为 0 (返回值)。
 - 将子进程加入进程列表, 状态设为 `PROC_RUNNABLE`。
 - 返回子进程 PID 给父进程。
- 返回: 父进程从 `sys_fork` 返回 PID。子进程被调度后, 从 `forkret` 开始, 最终返回 0。

2. exec:

- 用户态: 调用 `exec()` 系列函数, 执行 `ecall`。
- 内核态: `sys_exec -> do_execve`。
 - 检查并释放当前进程的内存空间 (`exit_mmap`, `put_pgdir`, `mm_destroy`)。

- `load_icode`: 加载新的 ELF 二进制文件，建立新的内存映射，设置新的 `trapframe`（如练习1所述）。
- 返回: 不返回原来的程序点，而是返回到新程序的入口点 (`elf->e_entry`) 开始执行。

3. wait:

- 用户态: 调用 `wait()`，执行 `ecall`。
- 内核态: `sys_wait` -> `do_wait`。
 - 查找状态为 `PROC_ZOMBIE` 的子进程。
 - 如果找到，释放子进程剩余的资源（内核栈、`proc_struct`），并获取子进程的退出码。
 - 如果没有 ZOMBIE 子进程但有运行中的子进程，将当前进程状态设为 `PROC_SLEEPING` 并调用 `schedule()` 让出 CPU。
- 返回: 返回子进程 PID 和退出状态。

4. exit:

- 用户态: 调用 `exit()`，执行 `ecall`。
- 内核态: `sys_exit` -> `do_exit`。
 - 释放大部分内存资源 (`exit_mmap`)。
 - 将状态设为 `PROC_ZOMBIE`。
 - 设置退出码。
 - 如果有父进程在等待 (`WT_CHILD`)，唤醒父进程。
 - 将所有子进程过继给 `init` 进程。
 - 调用 `schedule()`，不再返回。

内核态与用户态的交错执行:

程序正常在用户态执行。当需要操作系统服务（如 `fork`）或发生中断/异常（如页错误）时，硬件机制将特权级切换到 S 模式（内核态），跳转到内核的中断处理程序。内核处理完毕后，通过 `sret` 指令恢复上下文并返回用户态。

结果返回:

系统调用的返回值通常存放在寄存器 `a0` 中。内核在处理系统调用时，会将返回值写入当前进程 `trapframe` 的 `a0` 字段。当 `forkrets` 恢复寄存器时，用户程序就能在 `a0` 中看到返回值。

用户态进程执行状态生命周期图



- **PROC_UNINIT:** 进程刚被创建，尚未初始化完成。
- **PROC_RUNNABLE:** 进程已准备好运行，在就绪队列中等待调度。
- **PROC_RUNNING:** 进程正在 CPU 上执行（在 uCore 的简化模型中，RUNNING 状态通常隐含在 RUNNABLE 中，即 `current` 指向的 RUNNABLE 进程即为 RUNNING）。
- **PROC_SLEEPING:** 进程因等待某些事件（如子进程退出、IO）而挂起。
- **PROC_ZOMBIE:** 进程已退出，等待父进程回收资源。

扩展练习 Challenge

用户程序是何时被预先加载到内存中的？

在本次实验中，用户程序是在内核启动加载时，作为内核镜像的一部分被一同加载到物理内存中的。

具体机制分析：

通过查看 `Makefile` 可以发现，内核的链接规则如下：

```
$(kernel): $(KOBJS) $(USER_BINS)
    @echo + ld $@
    $(V)$(LD) $(LDFLAGS) -T tools/kernel.ld -o $@ $(KOBJS) --
format=binary $(USER_BINS) --format=default
```

这里 `$(USER_BINS)` 包含了编译好的用户程序二进制文件。链接器 `ld` 使用 `--format=binary` 选项将这些二进制文件直接链接到内核可执行文件 (`bin/kernel`) 中。链接器会自动为这些二进制文件生成起始地址符号（如 `__binary_obj__user_exit_out_start`）。

因此，当 Bootloader 将内核镜像加载到物理内存时，这些用户程序的二进制代码也随之被加载到了内存中。在 `kern/process/proc.c` 中，通过 `KERNEL_EXECVE` 宏直接引用这些内存地址来获取用户程序的代码。

与我们常用操作系统的加载有何区别，原因是什么？

区别：

- 本次实验中用户程序**静态链接**在内核镜像中，随内核一起加载到内存。它们驻留在物理内存中，不依赖文件系统进行存储和检索。
- 而常用操作系统中 (如 Linux/Windows): 用户程序通常存储在**磁盘**（或其他持久化存储设备）的文件系统中。只有当用户请求执行某个程序（例如调用 `exec` 系统调用）时，操作系统才会通过文件系统驱动程序解析可执行文件（如 ELF 或 PE 格式），将其从磁盘**动态加载**到内存中。通常还会利用**按需分页 (Demand Paging)** 机制，仅在访问特定页面时才从磁盘读取数据。

原因：

1. **简化实验设计**: 在 Lab5 阶段，实验的重点是进程管理（Process Management）和虚拟内存管理（Virtual Memory Management）。此时 uCore 尚未实现完善的文件系统，为了让用户程序能够运行并测试进程管理功能，将用户程序直接嵌入内核是最简单、最直接的方法，避免了引入文件系统的复杂性。
2. **嵌入式场景**: 这种做法在一些资源受限、没有文件系统的简单嵌入式系统或实时操作系统 (RTOS) 中也是存在的，称为 XIP (Execute In Place) 或直接将应用固化在 Flash 中。

分支任务：GDB调试

本练习旨在通过 GDB 动态调试工具，验证 uCore 操作系统在 RISC-V 架构下的系统调用处理流程及虚拟内存管理机制，并进一步深入 QEMU 模拟器源码，分析硬件行为的软件模拟实现。

4.1 GDB 调试流程设计与执行

依据实验指导手册中关于 GDB `eca11` 调试与 MMU 调试的要求，设计如下调试方案：

4.1.1 调试环境构建

为了在操作系统启动初期介入控制，需配置 QEMU 在 CPU 加载内核后立即挂起。

1. 修改启动参数: 在 `Makefile` 或 QEMU 启动脚本中添加 `-s -s` 选项。

- `-s`: 冻结 CPU，等待 GDB 指令。
- `-s`: 开启 GDB 服务器，默认监听 TCP 1234 端口。

2. 建立连接:

启动 QEMU 后，在另一终端运行 GDB 并加载内核符号表:

```
riscv64-unknown-elf-gdb bin/kernel
(gdb) target remote :1234
```

4.1.2 系统调用 (`eca11`) 的流程

本节验证用户态程序通过 `eca11` 指令陷入内核态，并由内核通过 `sret` 返回用户态的完整过程。

1. 定位系统调用入口:

在内核的陷入处理入口 `__alltraps` 或具体的系统调用分发函数 `syscall` 处设置断点。为观察特权级切换，建议在汇编级入口处中断。

```
(gdb) break __alltraps
(gdb) continue
```

2. 触发与观察:

运行触发系统调用的用户程序（如 `exit` 或 `fork`）。当程序在断点处停下时，执行以下检查:

- 检查陷入原因: 查看 `scause` 寄存器。

```
(gdb) info registers scause
```

对于用户态系统调用，`scause` 的值应为 8 (`User mode environment call`)。

- 检查上下文保存: 此时 `sscratch` 寄存器应保存了用户栈指针（或内核栈指针，取决于具体的上下文切换阶段），而 `sepc` 寄存器应指向触发异常的用户态 `ecall` 指令地址。

```
(gdb) info registers sepc sstatus
```

同时检查 `sstatus` 的 `SPP` 位，确认陷入前的特权级为 User Mode (0)。

3. 单步跟踪内核处理:

使用 `stepi` (`si`) 指令单步执行，观察 `__alltraps` 如何将通用寄存器保存至栈上的 `trapframe` 结构中。

```
(gdb) x/32x $sp # 查看栈上保存的 trapframe 内容
```

4. **验证中断返回（`sret`）**:

在 `__trapret` 标号处设置断点，观察恢复上下文的过程。执行 `sret` 指令后，GDB 无法直接跟踪（特权级切换），需在 `sepc` 指向的目标用户地址处预设断点，以验证 CPU 正确跳转回用户程序。

4.1.3 虚拟内存管理（MMU）的动态调试

本节验证页表映射的正确性及 TLB 的行为。

1. **获取页表基址**:

在分页机制开启后，通过 `satp` 寄存器获取根页表的物理页号（PPN）。

```
```gdb
```

```
(gdb) print/x $satp
```

### 2. QEMU Monitor 辅助检查:

利用 QEMU 自带的 Monitor 能够查看当前的 TLB 状态和页表树。在 GDB 中无法直接调用 `monitor` 命令时，可在 QEMU 窗口使用 `Ctrl+A, C` 切换，或使用 GDB 的 `monitor` 前缀指令（如支持）。

```
(gdb) monitor info mem
```

该指令将打印当前进程完整的虚拟地址空间映射表，包括权限位（R/W/X/U），用于验证代码段是否只读、数据段是否可写。

### 3. 手动验证地址转换:

选取一个有效的虚拟地址（如内核栈地址），手动模拟 Sv39 页表查找过程:

- 利用 GDB 的 `x` 命令读取物理内存中的页表项 (PTE)。

- 根据 VPN (Virtual Page Number) 索引逐级查找 L2 -> L1 -> L0 页表。
- 验证最终计算出的物理地址中的数据与直接访问虚拟地址获取的数据是否一致。

```
(gdb) x/x 0xfffffffffc0200000 # 访问虚拟地址
(gdb) # ... 根据 satp 计算物理地址 ...
(gdb) x/x <Calculated_Physical_Address>
```

## 4.2 指令模拟与地址翻译机制分析

通过上述 GDB 外部观测，结合 QEMU 源码分析，进一步揭示硬件行为的软件实现逻辑。

### 4.2.1 TCG (Tiny Code Generator) 翻译机制

QEMU 作为一个动态二进制翻译器，在执行 RISC-V 架构指令时，采用 TCG 机制进行转换：

1. **翻译 (Translation):** 将 RISC-V 目标指令解码并翻译为架构无关的 TCG 中间码 (IR)。
2. **执行 (Execution):** 宿主机 CPU 执行编译后的中间码，通过更新内存中的结构体（如 `CPURISCVState`）来模拟寄存器状态的变化。

### 4.2.2 特权指令的模拟流程

- **ecall 处理:**  
源码路径: `target/riscv/insn_trans/trans_privileged.inc.c`。  
当执行 `ecall` 时，调用 `helper_raise_exception`。该函数更新 `scause` 为 `RISCV_EXCP_U_ECALL`，更新 `sepc` 为当前 PC，并调用 `cpu_loop_exit` 中断当前执行流，模拟进入异常处理程序。
- **sret 处理:**  
源码路径: `target/riscv/op_helper.c (helper_sret)`。  
该函数读取 `sstatus` 中的 `SPIE` 和 `SPP` 位，恢复中断使能和特权级，并将 `pc` 重置为 `sepc` 的值，从而模拟硬件的中断返回。

### 4.2.3 MMU 与页表漫游 (Page Table Walk)

在 Sv39 分页模式下，QEMU 通过 `target/riscv/cpu_helper.c` 中的 `get_physical_address` 函数模拟硬件 PTW：

1. **循环遍历:** 代码中存在一个循环（`levels - 1` 到 `0`），模拟从根页表逐级向下查找的过程。

2. 物理内存读取: 使用 `address_space_ldq` 函数模拟读取物理内存中的 PTE。
3. 终止判断: 根据 PTE 的 R/W/X 权限位判断是否为叶子节点。若非叶子节点, 则提取 PPN 继续索引下一级页表。

此过程揭示了 QEMU 如何通过软件算法精确复现硬件的 MMU 逻辑。

## 4.3 内存管理单元 (MMU) 与页表漫游 (Page Table Walk)

在开启分页模式 (如 Sv39) 下, 虚拟地址到物理地址的转换由软件模拟的 MMU 完成。核心逻辑位于 `target/riscv/cpu_helper.c` 中的 `get_physical_address` 函数。

### 调试分析: 地址翻译流程

场景: uCore 执行访存指令, 触发 TLB 缺失, 进入页表查找逻辑。

关键代码逻辑分析 (Sv39模式):

`get_physical_address` 函数包含一个核心循环, 模拟硬件的页表漫游单元 (PTW):

```
/* 逻辑抽象 */
for (i = levels - 1; i >= 0; i--) {
 /* 1. 计算当前级页表项 (PTE) 的物理地址 */
 hwaddr pte_addr = (ppn << PGSHIFT) + ((vaddr >> (i * ptidxbits
+ PGSHIFT)) & 0x1ff) * sizeof(target_ulong);

 /* 2. 模拟物理内存读取, 获取 PTE 内容 */
 target_ulong pte = address_space_ldq(as, pte_addr, attrs,
&res);

 /* 3. 检查 PTE 有效位 (PTE_V) 与权限位 */
 if (!(pte & PTE_V)) {
 return TRANSLATE_FAIL; // 触发 Page Fault
 }

 /* 4. 判断是否为叶子节点 */
 /* 如果 R/W/X 位任意一位被置位, 表示找到物理页 (可能是大页或普通页) */
 if ((pte & (PTE_R | PTE_W | PTE_X)) != 0) {
 break; // 结束漫游
 }

 /* 5. 非叶子节点, 更新 PPN 指向下一级页表基址, 继续循环 */
 ppn = pte >> PTE_PPN_SHIFT;
}
```

关键操作说明：

- 多级遍历: 循环模拟了从根页表（基址由 `satp` 提供）逐级向下查找的过程。
- 物理内存读取: `address_space_ldq` 函数用于在模拟器层面读取客户机物理地址的数据。
- 终止条件: 当检测到 PTE 的读/写/执行位非空时，判定为叶子节点，终止循环并计算最终物理地址；否则继续根据 PPN 索引下一级页表。

## 4.4 QEMU TLB 模拟机制

### 1. TLB 查找代码路径

QEMU 使用软件定义的 TLB 结构来加速地址转换，避免频繁进行昂贵的页表漫游。

- 通用实现: `accel/tcg/cputlb.c`，核心函数为 `tlb_hit`（快速路径）。
- RISC-V 接口: `target/riscv/cpu_helper.c` 中的 `riscv_cpu_tlb_fill`。
- 执行逻辑:
  - a. CPU 发起访存，首先查询 `CPUTLBEntry` 结构（软件哈希表）。
  - b. 若发生 **TLB Miss**，调用 `riscv_cpu_tlb_fill`。
  - c. 在该函数中调用 `get_physical_address` 执行上述页表漫游。
  - d. 获取物理地址后，调用 `tlb_set_page` 将虚拟地址与物理地址（以及宿主主机虚拟地址）的映射关系填充回软件 TLB。

### 2. 模拟 TLB 与 硬件 TLB 的差异

- 硬件 **TLB**: 位于 CPU 内部的高速缓存（通常为相联存储器），支持并行查找。硬件 TLB 缺失后，由硬件 PTW 或软件异常处理程序填充。
- QEMU 软件 **TLB**:
  - 数据结构: 基于哈希表的 C 语言结构体数组。
  - 映射对象: 硬件 TLB 缓存 Guest VA 到 Guest PA 的映射；QEMU TLB 为了加速访存，同时缓存了 Guest PA 到 **Host VA** (宿主机虚拟地址) 的偏移量，以便宿主机 CPU 能直接通过指针操作访问模拟内存，而无需每次都进行软件层面的地址转换。
  - 处理逻辑: 在未开启分页（Bare Metal）模式下，QEMU 可能会跳过复杂的漫游逻辑，直接建立恒等映射，体现了模拟器针对不同模式的优化策略。

## 4.5 大模型辅助调试过程记录

在实验过程中，利用大语言模型（LLM）作为辅助工具，解决了源码定位与逻辑理解的难点。

### 问题 1: 源码定位困难

- **问题背景:** QEMU 源码结构复杂，大量使用宏定义与 `.inc.c` 包含文件，导致难以直接搜索到 `ecall` 指令的定义位置。
- **LLM 交互:** 询问 QEMU 中 RISC-V 特权指令的实现文件位置。
- **解决方案:** 根据模型提示，定位到 `target/riscv/insn_trans/trans_privileged.inc.c` 文件，并获知 `trans_ecall` 调用了 `helper_raise_exception`，从而快速找到了调试断点入口。

### 问题 2: 页表漫游逻辑解析

- **问题背景:** 在阅读 `get_physical_address` 函数时，对循环内部的位运算操作及 `address_space_ldq` 的作用存疑。
- **LLM 交互:** 提交代码片段，询问循环终止条件及访存函数的含义。
- **解决方案:** 模型解释了 Sv39 分页模式下 PTE 的格式，指出了 `break` 语句是基于 PTE 权限位判断叶子节点（区分大页与普通页），明确了 `address_space_ldq` 是模拟器读取客户机物理内存的接口。这有助于理解软件模拟硬件 PTW 的精确步骤。